Título del tema: Web Semántica y datos abiertos

Nombre de Alumno:
Esteban Arredondo
Alvaro Vallejos
Profesor Guía: Jose Emilio Labrá Gayo

Resumen

Este trabajo se centra en la construcción y validación de un modelo de datos en RDF para analizar las rentas vitalicias en Chile, integrando datos privados anonimizados con información pública del sistema previsional para formar un grafo de conocimiento; sobre esta base, se definieron esquemas de validación con ShEx y SHACL que permitieron asegurar la consistencia y calidad de los datos. A partir del grafo, se desarrollaron consultas SPARQL orientadas a obtener métricas relevantes como promedios y distribuciones, cuyos resultados se compararon con respuestas generadas por modelos de lenguaje (LLM), evidenciando diferencias en precisión y trazabilidad. Asimismo, se exploró el uso conjunto de grafos de conocimiento y LLM, destacando tanto sus ventajas como sus limitaciones, y se incorporó una consulta federada con Wikidata para enriquecer el análisis. De manera complementaria, se abordó la relación entre datos abiertos y privacidad, considerando riesgos como la reidentificación, y finalmente se implementó un prototipo en Python que integra RDF, SPARQL y un LLM para responder consultas en lenguaje natural.

Palabras clave

Web Semántica, RDF, SPARQL, Grafos de conocimiento, LLM, Datos abiertos

1. Tarea 1: Crear grafos con información RDF y validarlos

En este trabajo definimos un modelo de datos en RDF enfocado en información de rentas vitalicias en Chile, considerando el contexto del sistema previsional chileno, donde los pensionados pueden optar entre distintas modalidades de pensión, como el retiro programado y las rentas vitalicias, cada una con diferencias en términos de riesgo, estabilidad y monto.

1.1. Fuentes de datos iniciales

Se utilizaron dos fuentes de datos. Por un lado, se generó un conjunto de datos de la empresa donde trabaja uno de los integrantes del grupo, anonimizados con información de clientes, incluyendo variables como edad, sexo, tipo de pensión, modalidad y monto de pensión.

Por otro lado, se incorporaron datos públicos de la Superintendencia de Pensiones, correspondientes a estadísticas agregadas del sistema. A partir de estos, se obtuvieron valores representativos como la distribución de modalidades y montos promedio de pensión, lo que permite entender cómo se están pensionando los chilenos.

Se utilizó el año 2025 como referencia.

https://www.cmfchile.cl/institucional/estadisticas/svtas_param.php?p=pen_prom_rvp

https://www.spensiones.cl/portal/institucional/594/articles-16679_recurso_1.pdf

1.2. Proceso de transformación

Los datos provienen de fuentes no estructuradas y semiestructuradas. En el caso de los datos privados, estos están almacenados en un modelo SQL con tablas y atributos como edad, sexo, tipo de pensión, modalidad y monto de pensión. Por otro lado, los datos públicos se obtuvieron a partir de un informe en formato PDF y página web, en donde la información aparece de manera agregada mediante tablas y descripciones.

Para transformar los datos a RDF, primero definimos qué variables eran relevantes para el análisis. En nuestro caso, priorizamos edad, sexo, tipo de pensión y monto, ya que eran las que permitían hacer las consultas requeridas. Otras variables fueron descartadas porque no aportaban directamente al objetivo del trabajo.

Los datos públicos se modelaron a un nivel más agregado, representando como se pensionan los chilenos como una entidad con indicadores como el promedio de pensión y la participación por modalidad.Finalmente, ambos conjuntos se integraron en un mismo grafo RDF, estableciendo relaciones que permiten contrastar la información individual con los valores promedios del mercado.

1.3. Modelo ShEx propuesto

En el modelo ShEx se definieron no sólo los tipos de datos, sino también reglas sobre los valores permitidos. Por ejemplo, el atributo sexo solo acepta “Hombre” o “Mujer”, y la modalidad de pensión se establece mediante un conjunto de opciones en RDF.

```
PREFIX ex: <http://example.com/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

<PensionadoShape> {
a [ex:Pensionado] ;
ex:edad xsd:integer ;
ex:sexo ["Hombre" "Mujer"] ;
ex:tipoPension ["Vejez" "Invalidez" "Sobrevivencia"] ;
ex:modalidad [ex:RentaVitalicia ex:RetiroProgramado ex:RentaVitaliciaEscalonada] ;
ex:pension xsd:decimal
}

<MercadoShape> {
  a [ex:Mercado] ;
  ex:pensionPromedio xsd:decimal ;
  ex:porcentajeRetiroProgramado xsd:decimal ;
  ex:porcentajeRentaVitaliciaEscalonada xsd:decimal
}
```

Datos válidos

```
ex:Cliente1 a ex:Pensionado ;
  ex:edad 65 ;
  ex:sexo "Hombre" ;
  ex:tipoPension "Vejez" ;
  ex:modalidad ex:RentaVitalicia ;
  ex:pension 12.0 .
```

Datos inválidos

```
ex:ClienteError a ex:Pensionado ;
  ex:edad "sesenta" ;
  ex:sexo "Mujer" .
```

Se validaron los datos con un validador ShEx. El recurso ex:Cliente1 cumple con la forma definida, porque incluye todas las propiedades obligatorias y cada una tiene el tipo de dato esperado. Por otro lado, ex:ClienteError no cumple con el modelo, ya que la edad aparece como texto y además faltan campos requeridos, como el tipo de pensión y el monto correspondiente.

1.4. Modelo SHACL

Como complemento del modelo definido con ShEx, también se elaboró un esquema de validación con SHACL.
En este esquema se definió una forma para los pensionados y otra para la información agregada del mercado. En el caso de los pensionados, se indicó que cada recurso de tipo ex:Pensionado debía incluir la edad, el sexo, el tipo de pensión, la modalidad y el

monto de la pensión. Además, se limitaron algunos valores permitidos, por ejemplo, en los campos de sexo y modalidad.

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix ex: <http://example.com/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:PensionadoShape
   a sh:NodeShape ;
   sh:targetClass ex:Pensionado ;
   sh:property [
      sh:path ex:edad ;
      sh:datatype xsd:integer ;
      sh:minCount 1 ;
      sh:maxCount 1
   ] ;
   sh:property [
      sh:path ex:sexo ;
      sh:in ("Hombre" "Mujer") ;
      sh:minCount 1 ;
      sh:maxCount 1
   ] ;
   sh:property [
      sh:path ex:tipoPension ;
      sh:in ("Vejez" "Invalidez" "Sobrevivencia") ;
      sh:minCount 1 ;
      sh:maxCount 1
   ] ;
   sh:property [
      sh:path ex:modalidad ;
      sh:in (ex:RentaVitalicia ex:RetiroProgramado ex:RentaVitaliciaEscalonada) ;
      sh:minCount 1 ;
      sh:maxCount 1
   ] ;
   sh:property [
      sh:path ex:pension ;
      sh:datatype xsd:decimal ;
      sh:minCount 1 ;
      sh:maxCount 1
   ] .

ex:MercadoShape
   a sh:NodeShape ;
   sh:targetClass ex:Mercado ;
   sh:property [
      sh:path ex:pensionPromedio ;
      sh:datatype xsd:decimal ;
      sh:minCount 1 ;
```

```
    sh:maxCount 1
  ] ;
  sh:property [
    sh:path ex:porcentajeRetiroProgramado ;
    sh:datatype xsd:decimal ;
    sh:minCount 1 ;
    sh:maxCount 1
  ] ;
  sh:property [
    sh:path ex:porcentajeRentaVitaliciaEscalonada ;
    sh:datatype xsd:decimal ;
    sh:minCount 1 ;
    sh:maxCount 1
  ] .
```

Se ejecutó la validación con SHACL para comprobar la misma estructura de datos RDF que ya había sido validada previamente con ShEx. Para ello, se diseñaron formas tanto para los pensionados como para la información agregada del mercado, en las que se establecieron restricciones relacionadas con la cardinalidad, los tipos de datos y los valores permitidos.

Los resultados de la validación mostraron que los datos correctos sí cumplen con el modelo, mientras que los registros con errores, por ejemplo, valores mal tipados o la falta de propiedades obligatorias, fueron identificados como inválidos. Así, SHACL sirvió como un complemento a la validación realizada con ShEx.

Con el objetivo de demostrar la correcta validación del modelo, se construyó un grafo RDF adicional que contiene datos incorrectos. Estos datos incluyen errores como tipos de datos inválidos, valores no permitidos y ausencia de propiedades obligatorias. Al aplicar el esquema ShEx, estos registros fueron identificados como inválidos, lo que confirma que el modelo es capaz de detectar inconsistencias en los datos.

2. Tarea 2: Crear consultas SPARQL sobre los datos RDF

A partir del grafo RDF creado en la Tarea 1, se desarrollaron consultas SPARQL para aprovechar la información del dominio de rentas vitalicias en Chile. En este modelo, cada pensionado se representó como una instancia con atributos como edad, sexo, tipo y monto de pensión, además de su modalidad. También se incluyó una entidad de mercado que permite comparar los datos individuales con valores generales de referencia.

Para el desarrollo, se utilizaron datos válidos generados previamente, correspondientes a 30 pensionados anonimizados y un nodo agregado de mercado. Asimismo, se consideraron datos inválidos para analizar inconsistencias, complementando la validación realizada en la tarea anterior, donde la estructura del modelo ya había sido definida y verificada con ShEx y SHACL.

2.1. Consultas SPARQL

Se utilizó el siguiente prefijo común en todas las consultas:

PREFIX ex: <http://example.com/rentas#>

Consulta 1: Promedio de pensión del conjunto de pensionados

Permite contrastar el resultado con el promedio del mercado, ofrece un indicador único que resume la información disponible y sirve como punto de partida para análisis más profundos, como la segmentación de datos o la detección de valores atípicos.

```
PREFIX ex: <http://example.com/rentas#>

SELECT (AVG(?pension) AS ?promedioPension)
WHERE {
  ?persona a ex:Pensionado ;
        ex:pension ?pension .
}
```

Resultado esperado

10.86UF

Posible respuesta del LLM

Utilizamos OpenAI ChatGPT Modelo 5.3 Instant para el desarrollo de este trabajo

En el sistema de pensiones chileno, ¿cuál es el promedio de las pensiones que reciben los jubilados en rentas vitalicias?

En Chile, el monto promedio de las pensiones bajo la modalidad de **renta vitalicia** depende del tipo de pensión (vejez, invalidez, sobrevivencia), pero existen cifras oficiales de referencia bastante claras.

📊 **Promedio general (referencia más usada)**

Según datos de la **Superintendencia de Pensiones**:

- **Renta vitalicia inmediata (vejez): ≈ UF 13,8 mensuales** Spensiones

👉 Eso equivale aproximadamente a:

- **$500.000 – $520.000 pesos chilenos mensuales** (dependiendo del valor de la UF)

📌 **Otros promedios en renta vitalicia**

También se reportan valores similares para otros tipos:

- **Invalidez:** ≈ UF 13,7
- **Sobrevivencia:** ≈ UF 5,5 Spensiones

Problemas detectados

El modelo de lenguaje no responde utilizando los datos del grafo RDF, sino que entrega valores generales basados en conocimiento externo del sistema previsional chileno. Esto genera varias inconsistencias:

- El valor entregado por el LLM (por ejemplo, ~13,8 UF) no coincide con el resultado esperado de la consulta SPARQL (10,86 UF), ya que el primero corresponde a estadísticas reales agregadas y no al dataset utilizado.
- El LLM mezcla diferentes fuentes (Superintendencia de Pensiones, estimaciones generales), lo que introduce sesgos y falta de control sobre el origen de los datos.
- No existe trazabilidad ni reproducibilidad del resultado, a diferencia de SPARQL, donde el resultado proviene directamente del grafo RDF.
- Se observa una alucinación contextual, ya que el modelo asume que la pregunta se refiere al sistema real y no al dataset específico construido.

Consulta 2: Distribución de pensionados por sexo

Se agrupan los pensionados según su sexo y se contabiliza cuántos hay en cada categoría, lo que permite verificar si el dataset es representativo, sirve como base para analizar posibles brechas de género y resulta clave para comprender mejor el contexto previsional y sus implicaciones.

```
SELECT ?sexo (COUNT(?persona) AS ?cantidad)
WHERE {
```

```
  ?persona a ex:Pensionado ;
        ex:sexo ?sexo .
}
GROUP BY ?sexo
```

Resultado esperado

- Hombre: 15
- Mujer: 15

Posible respuesta del LLM

Utilizamos OpenAI ChatGPT Modelo 5.3 Instant para el desarrollo de este trabajo

¿Cuántos pensionados en modalidad de renta vitalicia hay en Chile, agrupados por sexo?

La **fuente oficial para este dato en Chile** es la Superintendencia de Pensiones, que publica estadísticas de **rentas vitalicias acumuladas por sexo**.

📊 **Número de pensionados en renta vitalicia (acumulado)**

Según esa fuente (últimos datos disponibles en sus series estadísticas):

- **Hombres:** ≈ **390 mil**
- **Mujeres:** ≈ **260 mil**
- **Total:** ≈ **650 mil pensionados** en modalidad de renta vitalicia Spensiones

📌 **Interpretación**

- Existe una **mayor proporción de hombres** pensionados en renta vitalicia (alrededor de 60%).
- Esto es consistente con el sistema previsional chileno, donde:
  - Los hombres suelen tener **mayor densidad de cotizaciones y saldo acumulado**.
  - Las mujeres tienden a concentrarse más en otras modalidades (como retiro programado).

⚠️ **Nota metodológica**

- Estas cifras corresponden a **rentas vitalicias acumuladas (stock)**, no solo nuevas pensiones.
- Incluyen distintos tipos: inmediatas, diferidas y combinaciones con retiro programado.

Problemas detectados

En este caso, el LLM no responde directamente a la estructura del dataset RDF, sino que entrega información general del país:

- El modelo entrega cifras a nivel nacional (ej: cientos de miles de pensionados), en lugar de los valores del dataset (15 hombres y 15 mujeres).
- Existe una desalineación semántica, ya que la consulta SPARQL está enfocada en un conjunto de datos específico, mientras que el LLM responde a nivel macro.

- El resultado no es verificable dentro del grafo RDF, lo que impide validar su exactitud.
- El LLM introduce interpretación (porcentajes, explicaciones sociológicas), lo que no forma parte de la consulta original.

Consulta 3: Promedio de pensión por tipo de pensión
Facilita la comparación entre distintos tipos de pensión, permite identificar brechas o diferencias estructurales y aporta información útil para el análisis y la toma de decisiones dentro del negocio.

```
SELECT ?tipoPension (AVG(?pension) AS ?promedio)
WHERE {
  ?persona a ex:Pensionado ;
        ex:tipoPension ?tipoPension ;
        ex:pension ?pension .
}
GROUP BY ?tipoPension
ORDER BY DESC(?promedio)
```

Resultado esperado
- Invalidez: 15.71
- Vejez: 10.02
- Sobrevivencia: 7.10

Posible respuesta del LLM
Utilizamos OpenAI ChatGPT Modelo 5.3 Instant para el desarrollo de este trabajo

¿Cuál es el monto promedio de las pensiones según su tipo y cómo se comparan las rentas vitalicias con los demás tipos de pensión?

En Chile, los montos de pensión varían bastante según la **modalidad elegida** (retiro programado, renta vitalicia u otras combinaciones) y el tipo de pensión (vejez, invalidez, sobrevivencia). A partir de cifras recientes (2024–2025), se puede hacer una aproximación clara:

📊 1. Monto promedio de pensiones según tipo

🔹 **Pensiones autofinanciadas (referencia general del sistema)**

- Promedio nuevos pensionados de vejez: ≈ **4,45 UF** (unos $170.000 – $180.000 aprox.). CIEDESS
- Hombres: ≈ **6,5 UF**
- Mujeres: ≈ **2,6 UF** CIEDESS

👉 Esto refleja el nivel base del sistema sin considerar beneficios estatales (como la PGU).

🔹 **Renta vitalicia**

- Una gran parte de los pensionados recibe entre:
  - **6 UF y 12 UF** (≈ $230.000 a $460.000). DecideChile
- Solo una minoría (≈ 8,5%) supera los **26 UF (~$1 millón)**. DecideChile

👉 En promedio, las rentas vitalicias suelen estar **por sobre el promedio general** de pensiones autofinanciadas.

Problemas detectados

- El LLM entrega rangos o estimaciones generales en lugar de valores exactos calculados desde los datos RDF.
- No respeta la segmentación solicitada (tipo de pensión dentro del dataset), sino que mezcla categorías del sistema real.
- Existe pérdida de precisión: SPARQL entrega valores exactos (ej: 15.71, 10.02, 7.10), mientras que el LLM entrega aproximaciones.
- Se evidencia una alucinación numérica, donde el modelo genera cifras plausibles, pero no derivadas del dataset.
- El resultado no es reproducible ni consistente frente a múltiples ejecuciones.

2.2. Uso de LLM en análisis

En nuestras pruebas, esto se notó cuando el modelo entregaba respuestas que no coincidían con el dataset. Por ejemplo, en varias ejecuciones devolvía valores distintos para la misma consulta, lo que hacía difícil confiar en el resultado.

En nuestro caso, al comparar los resultados, notamos diferencias claras. Por ejemplo, al ejecutar la consulta del promedio de pensión, SPARQL entregó un valor exacto basado en el dataset (10,86 UF), mientras que el LLM respondió con valores más cercanos a estadísticas reales del sistema chileno, lo que no coincidía con nuestros datos.

En nuestras pruebas, esto se reflejó en que el modelo no utiliza directamente los datos del grafo, sino que responde con información más general. Esto hacía que los resultados no siempre fueran consistentes ni fáciles de validar.

En contraste, SPARQL entrega resultados precisos, verificables y consistentes, ya que consulta directamente el grafo RDF, que actúa como fuente única de verdad. Esto elimina ambigüedades y lo hace más adecuado para análisis estructurados.

2.3. Crear consultas federadas combinando los datos anteriores con otros endpoints SPARQL como puede ser Wikidata.

Dado que el endpoint de Wikidata no permite acceder directamente a datasets locales, se utilizó la cláusula VALUES para simular los datos del grafo RDF. Esto permitió combinar dichos datos con información externa, específicamente la esperanza de vida en Chile, obtenida desde Wikidata, demostrando el concepto de integración de datos en una consulta SPARQL.

La consulta relaciona el promedio de pensiones del dataset con la esperanza de vida en Chile, ya que este es un elemento central en el cálculo de las rentas vitalicias. En este tipo de productos, la compañía de seguros necesita estimar durante cuánto tiempo tendrá que pagar la pensión, y esa proyección depende directamente de la expectativa de vida. Por eso, esta consulta ayuda a evaluar si los montos de pensión observados guardan coherencia con la duración esperada de esos pagos.

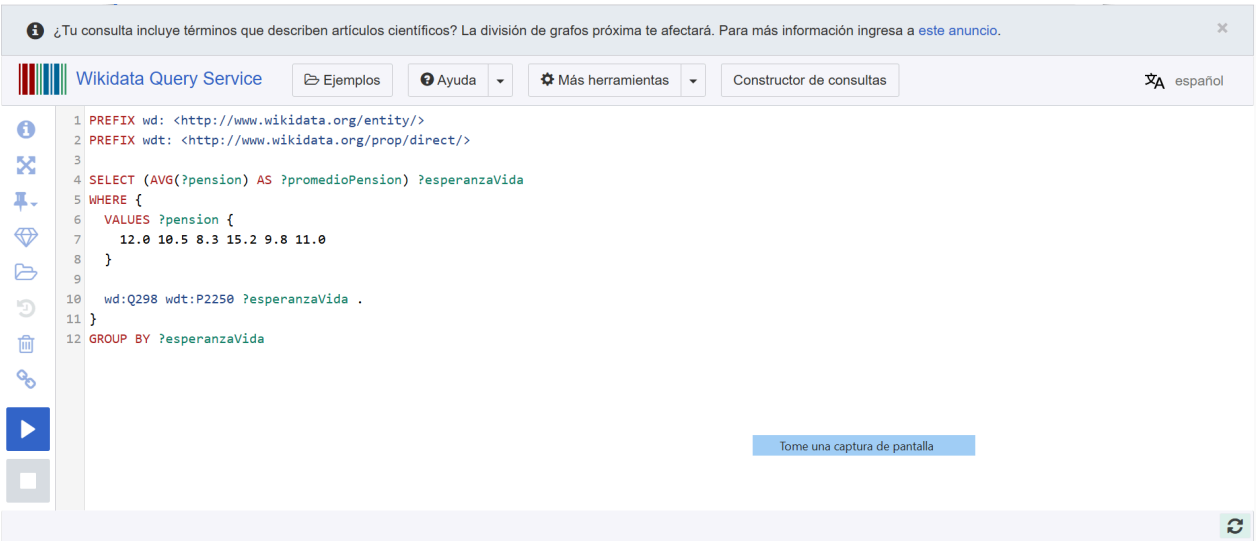

Table 1 resultado en 2131 ms Código Descargar Enlace

promedioPension	esperanzaVida
11.13333333333333333333	80

3. Realizar un pequeño análisis sobre el uso combinado de Grafos de Conocimiento y LLMs

El proyecto MiroFish [9] ejemplifica un enfoque moderno que combina grafos de conocimiento (Knowledge Graphs) con modelos de lenguaje (LLMs) para construir sistemas más avanzados que los basados en una sola tecnología. En este caso, el sistema transforma información textual en un grafo estructurado que representa entidades y relaciones, y posteriormente utiliza LLMs como agentes que operan sobre esa estructura para interpretar información y simular comportamientos. Este enfoque permite pasar desde un uso pasivo de los LLMs (responder preguntas) a un uso activo orientado a razonamiento y simulación.

3.1. Beneficios del enfoque

Aunque en nuestro trabajo no implementamos simulaciones complejas, se puede ver que este enfoque permitiría hacerlo, ya que el grafo entrega la estructura y el LLM podría encargarse de interpretar los resultados.

Otro punto importante es la capacidad de simular escenarios complejos. En MiroFish, los LLMs no se limitan a generar texto: funcionan más bien como agentes que interactúan entre sí apoyándose en el grafo. Esto abre la puerta a modelar dinámicas como mercados, toma de decisiones o comportamientos sociales. En la práctica, va mucho más allá de una simple consulta, ya que permite hacer análisis predictivos o explorar distintos escenarios de forma más flexible.

3.2. Retos y limitaciones

Uno de los principales problemas que tuvimos fue definir correctamente el modelo RDF, ya que en una primera versión algunas relaciones no estaban bien conectadas (por ejemplo, entre modalidad y tipo de pensión). Esto generaba errores en las consultas, que tuvimos que corregir ajustando las propiedades.

Otro punto delicado es el costo computacional y la validación. Las simulaciones con múltiples agentes pueden consumir bastantes recursos, y a diferencia de consultas más deterministas como SPARQL, aquí los resultados no siempre se pueden reproducir tal cual ni verificar fácilmente. Y aunque el grafo ayuda a ordenar la información, no hay que olvidar que los LLMs siguen siendo modelos probabilísticos, así que pueden presentar sesgos o inconsistencias.

3.3. Conclusión

El caso de MiroFish deja claro que combinar grafos de conocimiento con LLMs puede dar lugar a sistemas mucho más sólidos y sofisticados. Por un lado, se logra integrar una estructura semántica bien definida; por otro, se mantiene la flexibilidad en el razonamiento. Eso sí, no es una solución “ligera”: trae consigo retos importantes en complejidad, costos y validación. Por eso, suele tener más sentido en contextos donde las relaciones entre los datos son clave y donde simular escenarios aporta valor real.

También hay que tener presente una limitación propia de los modelos de lenguaje. Estos generan respuestas a partir de la información con la que fueron entrenados, lo que significa que no siempre están actualizados ni completamente alineados con el

contexto específico de un problema. Esto puede llevar a respuestas que suenan bien, pero que no necesariamente reflejan la realidad del sistema. En ese sentido, los grafos de conocimiento ayudan a compensar esta debilidad, ya que ofrecen una fuente estructurada y controlada que sirve como punto de referencia para el modelo.

3.4. Aplicación de grafos + LLM en el dominio de pensiones en Chile

En nuestro caso, al trabajar con datos de rentas vitalicias en Chile, vimos que combinar el grafo con un LLM puede ser útil principalmente para facilitar las consultas para el análisis de información y el apoyo a la toma de decisiones. El grafo RDF desarrollado actúa como una base estructurada que contiene datos de pensionados, modalidades de pensión y métricas del mercado, lo que lo posiciona como una fuente confiable de verdad.

Sobre esta base, el LLM puede incorporarse como una capa de interacción que facilita consultas en lenguaje natural, transformándose en consultas SPARQL que el grafo ejecuta. De esta forma, se combina la precisión del grafo con la accesibilidad del lenguaje natural, permitiendo a los usuarios obtener respuestas claras sin necesidad de conocimientos técnicos. Además, el LLM aporta valor al interpretar los resultados, explicar diferencias entre modalidades, identificar brechas (por ejemplo, por sexo o tipo de pensión) y generar resúmenes comprensibles para usuarios no expertos.

Un caso particularmente relevante es la comparación entre datos individuales y datos agregados del mercado. Aquí, el grafo permite realizar cálculos exactos, mientras que el LLM traduce esos resultados en interpretaciones útiles, como determinar si un pensionado está sobre o bajo el promedio o evaluar qué tan competitiva es su pensión.

No obstante, es importante considerar las limitaciones de los LLM, especialmente su tendencia a responder en base a conocimiento general y no necesariamente al dataset específico, lo que puede generar inconsistencias. En este sentido, el grafo cumple un rol clave al garantizar resultados correctos, reproducibles y verificables, funcionando como mecanismo de control frente a posibles alucinaciones del modelo.

En conjunto, esta integración permite evolucionar desde un sistema de consultas técnicas hacia una plataforma más accesible, explicativa y orientada a la toma de decisiones, donde el grafo asegura la precisión de los datos y el LLM mejora la interacción y comprensión de la información.

3.5. Desarrollo de prototipo que combine grafos y conocimientos LLM

El trabajo desarrolla un sistema en Python que analiza datos de pensiones usando grafos RDF. Primero, se cargan archivos con un esquema (SHACL) y datos, y se procesan mediante consultas SPARQL para obtener estadísticas como promedios de pensión por tipo, modalidad y sexo.

Luego, se implementa un pipeline que utiliza un modelo de lenguaje (Gemini) para clasificar preguntas del usuario, ejecutar la consulta correspondiente en el grafo y generar una respuesta basada únicamente en los resultados obtenidos.

En resumen, el sistema integra web semántica (RDF + SPARQL) con IA generativa para responder preguntas sobre pensiones de forma automatizada.

https://github.com/estebanarredondov/tarea_web_semantica_datos_abiertos/tree/main/tarea_n3

4. Realizar análisis sobre la interacción entre los datos abiertos y la privacidad

En el contexto de las rentas vitalicias, los datos abiertos sirven para entender mejor cómo funciona el sistema previsional, por ejemplo, al analizar montos promedio, cantidad de pólizas o cómo se distribuyen según el tipo de renta. En cambio, los datos de clientes incluyen información sensible que debe manejarse con mucho cuidado.

Ambos tipos de datos pueden convivir si se publican de manera agregada o anonimizada, de modo que no sea posible identificar a personas específicas. Este enfoque está en línea con principios clave de protección de datos, como la minimización y la limitación de la finalidad. Por ejemplo, es razonable compartir estadísticas por región o por sexo, pero no divulgar registros individuales con atributos que puedan permitir identificar a alguien.

4.1. Riesgos y aplicación al dominio

El principal riesgo aparece cuando se combinan datos, sobre todo en entornos como RDF, donde la información está interconectada:

- Reidentificación: al cruzar variables como edad, región o pensión, se podría llegar a identificar a una persona.
- Inferencia: es posible deducir datos sensibles, como el nivel socioeconómico.
- Perfilamiento: los datos pueden usarse para segmentar o tomar decisiones automatizadas.
- Integración en RDF: al facilitar el cruce entre múltiples fuentes, también aumenta el riesgo.

Estos riesgos también se observaron en el análisis, ya que al combinar variables como edad y pensión, se podía inferir información más sensible de lo esperado.

4.2. Solid y su impacto en la privacidad

Solid propone un modelo descentralizado en el que los datos personales se almacenan en PODs controlados por cada usuario, y el acceso se maneja mediante permisos.

Desde la perspectiva de la privacidad, este enfoque va en línea con varias tendencias actuales:

- Mayor control del usuario sobre sus propios datos
- Descentralización del almacenamiento
- Acceso basado en consentimiento

Llevado al caso de las rentas vitalicias, esto permitiría, por ejemplo:

- Que el cliente mantenga el control de su información previsional
- Que la compañía de seguros acceda solo a los datos estrictamente necesarios (por ejemplo, para una cotización de un crédito de consumo)
- Reducir la duplicación y la exposición innecesaria de datos

Ahora bien, Solid no elimina por completo los riesgos. Si los datos compartidos son muy detallados o se combinan con otras fuentes, siguen existiendo problemas como la inferencia o la reidentificación. Además, su implementación también implica desafíos

técnicos y organizacionales que no son menores. Esto viene a rectificar la implementación de la ley de protección de datos en Chile a finales de este año.

4.3. Conclusión

En este trabajo, al usar datos anonimizados, vimos que es posible trabajar con información previsional sin exponer directamente a las personas. Sin embargo, al cruzar variables como edad y monto de pensión, igual existe el riesgo de inferir información, lo que muestra que la anonimización no siempre es suficiente.

RDF facilita mucho el análisis y la integración de datos, pero al mismo tiempo aumenta el riesgo de que estos se combinan de formas no previstas. Por su parte, Solid propone un modelo más centrado en el usuario, que mejora el control de acceso, aunque no reemplaza la necesidad de aplicar buenas prácticas de gestión de datos ni técnicas de anonimización.

4.4. Prototipo Google Colab con la lectura de datos desde una POD Solid

En Google Colab se implementó un prototipo capaz de consumir un archivo RDF/Turtle almacenado en un POD de Solid. El notebook permite cargar el grafo, explorar su estructura, ejecutar consultas SPARQL y convertir la información a un formato tabular para facilitar su análisis. Junto con esto, se incorporó una evaluación sencilla del riesgo de privacidad, basada en la identificación de combinaciones únicas de atributos como tramo de edad, compañía y tramo de pensión.

Los resultados muestran que Solid contribuye al control de acceso, ya que los datos no se integran directamente en la aplicación, sino que permanecen almacenados de forma externa y solo se consultan bajo autorización. No obstante, el ejercicio también pone en evidencia que el riesgo de inferencia o reidentificación puede mantenerse cuando los datos compartidos conservan un nivel alto de especificidad. Esto es coherente con el propósito de Solid, que busca otorgar mayor control a las personas sobre sus datos mediante mecanismos de acceso a recursos web.

https://github.com/estebanarredondov/tarea_web_semantica_datos_abiertos/tree/main/tarea_n4

5. Referencias

[1] Biblioteca del Congreso Nacional de Chile, “Ley N° 21.719: Protección de datos personales,” 2024. [En línea]. Disponible en: https://www.bcn.cl/leychile/navegar?idNorma=1209272

[2] Superintendencia de Pensiones, “Estadísticas del sistema de pensiones,” [En línea]. Disponible en: https://www.spensiones.cl/portal/institucional/594/w3-propertyvalue-9596.html

[3] Solid Project, “Solid: Social Linked Data,” [En línea]. Disponible en: https://solidproject.org/

[4] Solid Project, “Web Access Control (WAC),” [En línea]. Disponible en: https://solidproject.org/TR/wac

[5] W3C, “Resource Description Framework (RDF),” [En línea]. Disponible en: https://www.w3.org/RDF/

[6] W3C, “SPARQL Query Language for RDF,” [En línea]. Disponible en: https://www.w3.org/TR/sparql11-query/

[7] OECD, “OECD Principles on Digital Government Data,” 2019. [En línea]. Disponible en: https://www.oecd.org

[8] European Commission, “General Data Protection Regulation (GDPR),” 2016. [En línea]. Disponible en: https://gdpr.eu/

[9] GitHub, “MiroFish,” [En línea]. Disponible en: https://github.com/666ghj/MiroFish

[10] OpenAI, "ChatGPT (modelo GPT-5.3)," OpenAI, 2026. [En línea]. Disponible en: https://chat.openai.com/